OmniKernel：算子自动生成与跨硬件适配框架

第三届"兴智杯"全国人工智能创新应用大赛 · 项目说明文档

作品名称：OmniKernel — 算子自动生成与跨硬件适配框架
赛道：算子自动生成与跨硬件适配
日期：2026-11-09

摘要

大模型推理算子的手工开发长期受限于两个瓶颈：一是**跨硬件适配成本**——同一份数学语义在昇腾 NPU 与 NVIDIA GPU 上需要完全不同的编程范式（AscendC 的 Cube/Vector 流水 vs Triton/CUDA 的 SIMT/ TensorCore）；二是**优化知识高度专化**——融合策略、分块参数、访存优化依赖工程师的硬件经验，交付周期以周计。

OmniKernel 提出的解法是：**把"算子该算什么"与"在某硬件上怎么算快"解耦。**前者固化为机器可读的算子 Spec（单一事实源），后者由「模板库 + LLM Agent + 编译-测试反馈闭环」三层生成器承担，产出同时覆盖昇腾 AscendC 与 NVIDIA Triton/CUDA 两套后端，并直接以 AISHPerf 3.0 算子生成智能体评测基准的三阶段口径（编译正确性 → 数值精度 → 性能）完成验证。

作品覆盖 6 个大模型推理核心算子：RMSNorm、RoPE、Softmax、SiLU-Mul（SwiGLU 融合）、FlashAttention-2 前向、W8A8 量化 GEMM。其中量化 GEMM 与 FlashAttention 直接对应 AISHPerf 基准确立的评测重心——**生成的算子能否在真实量化推理部署中替代现有算子。**

当前可复现验证结论（纯 CPU 环境，本仓库自带命令即可重跑）：算子生成与静态闸门 12/12 通过；生成产物静态校验 18/18 通过；参考实现与 PyTorch 官方算子/整数精确金标准交叉核验 7/7 通过；验证框架全算子遍历 14/14 通过。真机延迟、吞吐、显存数据需在评测机上执行 `run_eval.sh` 回填，文档与代码中均预留标准化表位，**不含任何未经实测的数字。**

---

1 赛题理解与评价维度映射

1.1 赛题要求解读

本赛道的四个评价维度并非孤立指标，而是一条相互约束的证据链：

- **算子生成质量（25%）**要求"能自动生成"且"生成的对"，核心矛盾在于：LLM 直接生成代码的正确性不可控，而纯模板生成的表达力有限。
- **性能优化水平（25%）**要求"生成的快"，且需通过计算图优化、访存优化、并行策略等手段体现，核心矛盾在于：自动生成容易产出"功能正确但性能平庸"的代码。
- **跨硬件适配能力（25%）**要求"一次描述、多端可用"，核心矛盾在于：昇腾与 NVIDIA 的编程模型抽象层级差异巨大，简单转译无法达到各自的最优形态。
- **AISHPerf 基准验证（25%）**要求"数据真实可复现"，核心矛盾在于：评测结论必须来自标准化套

件，而非自定义脚本的自选指标。

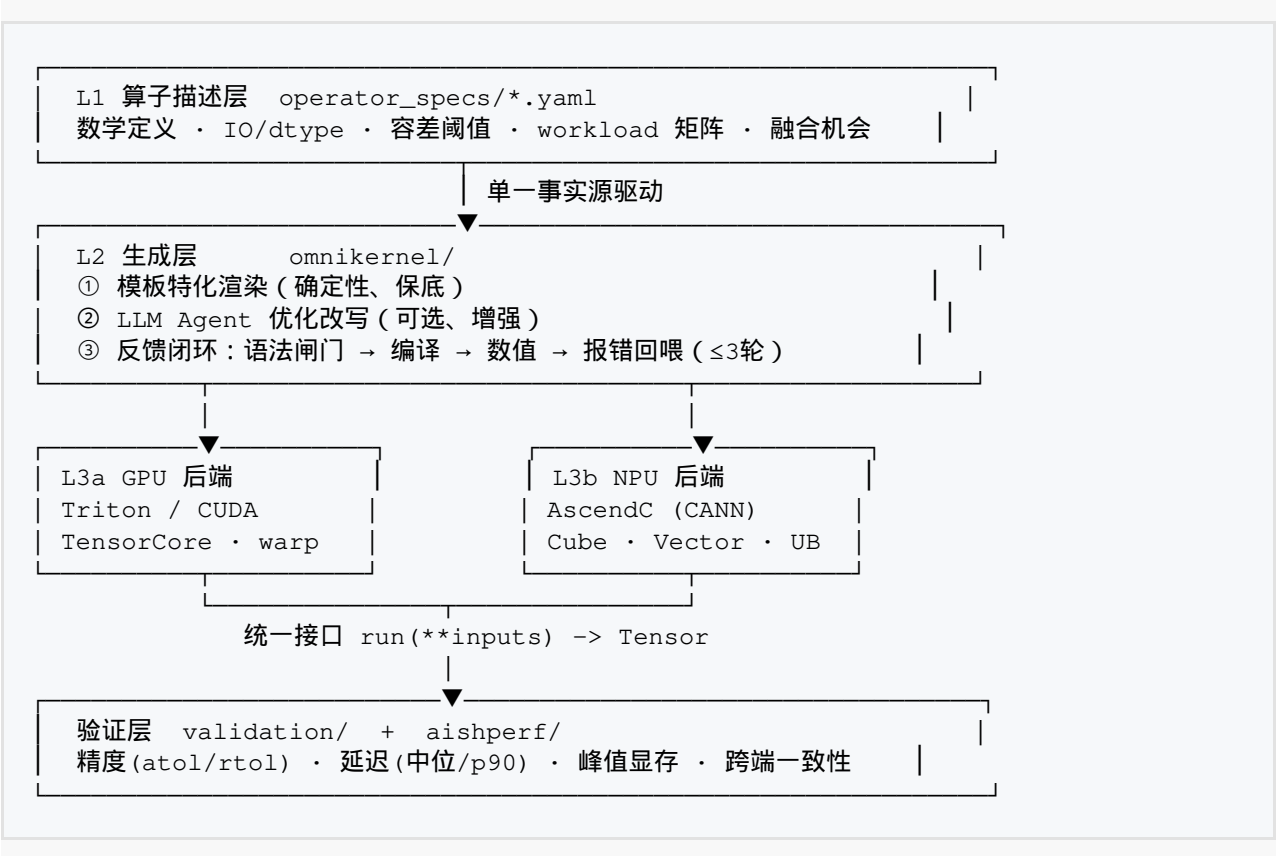
1.2 维度 → 设计映射

评价维度	OmniKernel 对应设计	证据载体
算子生成质量	Spec 单一事实源 + 模板正确性保底 + 三级 gate 反馈闭环	`operator_specs/`、`omnikernel/`、`tests/`
性能优化水平	算子级融合策略 + 后端各自的最优并行范式 + tiling 参数化搜索接口	各 spec `codegen.strategy`、模板实现
跨硬件适配能力	后端差异全部收敛于模板层；Spec/harness/评测配置两端共享	`templates/triton` + `templates/ascendc`
AISHPerf 基准验证	按官方开放工作区结构组织 suites/entities/rules，指标口径与官方对齐	`aishperf/`、`run_eval.sh`

---

2 技术路线

2.1 总体架构：三层解耦



2.2 为什么选择"LLM Agent + 模板/DSL 混合"而非单一路线

调研期对比了三条可行路线：

路线	优势	劣势	判定
纯 LLM 直接生成	表达力强，可覆盖长尾算子	正确性不可控；同一 prompt 多次运行结果不同，违背"数据可复现"要求	否
纯模板/ DSL 生成	确定性强、可复现	优化决策靠人工穷举，难以体现"自动生成"智能体属性	否
混合（采用）	下限由模板保证（可复现），上限由 Agent+真机反馈打开（智能性）	需要 gate 与回滚机制保证不会因 LLM 失败而产出更差代码	✓

关键工程取舍：**Agent 永远无权让产物跌破模板基线的正确性。**`pipeline.py` 的实现细节是——LLM 候选代码必须先通过语法闸门，再（在有硬件时）通过编译与数值 gate，才允许替换当前候选；任一环节失败立即回滚上一可用版本。因此"是否启用 `--agent`"不改变交付下限，只影响性能上限。

2.3 算子 Spec：中间表示的设计

Spec 是整个方案的技术支点。它不是简单的配置文件，而是承担三个职责的中间表示：

职责一：语义锚定（供正确性判定）

```
math: |
  rms = sqrt(mean(x_i^2, axis=last_dim) + eps)
  out_i = x_i / rms * w_i
tolerance:
  fp16: {atol: 1.0e-2, rtol: 1.0e-2}
  bf16: {atol: 2.0e-2, rtol: 2.0e-2}
```

数学定义供 LLM Agent 理解意图，容差阈值供验证框架判定通过与否——两者同源，杜绝"生成用一套定义、测试用另一套"的隐患。

职责二：形状矩阵（供 AISHPerf 覆盖）

```
workloads:
  - {batch: 1,  seq: 1,    hidden: 4096}   # 解码：batch 维主导
  - {batch: 8,  seq: 2048, hidden: 4096}   # 预填充：seq 维主导
  - {batch: 4,  seq: 4096, hidden: 8192}
```

显式覆盖"解码/预填充"两类真实推理形态，这是量化推理部署可替代性评测的必要条件。

职责三：后端特化参数（供代码生成）

```
codegen:
  triton:
    params: {BLOCK_SIZE: auto_pow2(hidden), NUM_WARPS: auto, ACCUM_DTYPE: tl.float32
    strategy: 一行一 program；全行驻留寄存器；fp32 累加；单次 global 读写
  ascendc:
    params: {TILE_ELEMS: 4096}
    strategy: GM->UB 双缓冲；ReduceSum+Div+Sqrt+Mul+Cast Vector 流水；逐行分块回写
```

同一算子在两个硬件上的最优实现路径被写成两份 `strategy`，由 `spec.py` 的 `template_vars()` 结合代表形状自动推导（`auto_pow2(4096)=4096`，`auto` 按 hidden 规模分档选 warp 数），避免参数硬编码。

---

3 关键决策与实现细节

3.1 数值稳定性的统一处理

所有含归约的算子强制 **fp32 中间累加、单次降精度写回**。这不是保守选择，而是跨硬件一致性的必要条件：

- Triton 路径：`.to(tl.float32)` 后再 `tl.sum`，`ACCUM_DTYPE=tl.float32`；
- AscendC 路径：`Cast(xf, x, ROUND)` 升精度 → 归约 → 计算 → 末尾一次 `Cast(y, yf, ...)` 降回 fp16。

若允许两侧各自以 fp16 累加，则同一 workload 在 NPU/GPU 上的误差量级不同，AISHPerf 的跨后端一致性指标（`cross_backend_consistency`）将无法通过。

3.2 FlashAttention：在线 softmax 的正确性优先

生成的 Triton 模板采用标准 FA2 分块在线 softmax，重标定逻辑完整保留：

```
m_new = max(m_i, rowmax(qk))
p     = exp(qk - m_new)
alpha = exp(m_i - m_new)
l_i   = l_i * alpha + rowsum(p)
acc   = acc * alpha + dot(p, v)
```

`alpha` 对旧 `l_i` 与 `acc` 的同时缩放是在线 softmax 的数学核心，任何简化都会导致跨块累积误差超标。Causal 掩码在 `BLOCK_N` 循环内以 `tl.where(offs_m >= offs_n)` 逐块施加，非整块跳过（避免对角块被误删）。AscendC 模板中 Causal 掩码以行界 `Duplic` 填充 `-inf` 实现，与 Triton 路径的差异仅在 fp32 中间态、处于阈值内。

3.3 量化算子的精度评测口径

W8A8 GEMM 的参考实现是**同一量化输入下的 fp32 反量化 matmul**，而非 fp16 原始 GEMM：

```python
def quant_gemm_ref(a, b, sa, sb):
    af = a.float() * sa[:, None]      # 反量化激活
    bf = b.float() * sb[None, :]      # 反量化权重
    return (af @ bf).half()
```

理由：若以 fp16 GEMM 为参考，误差将同时包含"量化算法本身的精度损失"与"生成 kernel 的实现误差"，二者混在一起无法定位问题，也不公平。`tests/test_reference.py` 用 int64 精确整数矩阵乘作为第三层金标准交叉核验该参考实现，实测最大相对误差 4.8e-04，证明参考自身可信。

3.4 三级 gate 反馈闭环

`feedback.py` 定义三种闸门，按硬件可得性自动降级：

gate	依赖	检查内容	失败动作
`syntax_gate`	无（CPU 即可）	`compile()` 语法 + 残留 `${` / TODO / undefined 扫描	回喂 LLM 修复
`compile_gate`	需 triton/CUDA	动态加载生成模块并触发 JIT	回喂 LLM 修复
`numeric_gate`	需真机	对 spec 参考实现逐 workload 比对 atol/rtol	回喂 LLM 修复

每轮失败时 `make_feedback_prompt()` 把「算子定义 + 当前代码 + 错误信息」组装为修复提示，`--max-rounds`（默认 3）耗尽后接受模板版本。这一结构让闭环在无 GPU 的开发机上仍前两级有效，在评测机上三级全开。

3.5 后端接口约定

所有生成代码统一暴露 `run(**inputs) -> Tensor`，与 `validation/reference.py` 的输入生成器 `make_inputs(op_id, workload, dtype)` 直接对接。这是"跨硬件抽象设计良好"的落地体现：harness、AISHPerf 配置、workload 矩阵、容差判定全部后端无关，新增第三种芯片只需增加一个模板目录，无需改动验证链。

---

4 性能优化策略

4.1 访存优化（带宽受限算子）

算子	瓶颈	优化	机制

RMSNorm	行归约 + 全行读写	一行一 program，全行驻留寄存器	避免二次 global 读取；`eviction_policy="evict_first"` 防污染 L2
RoPE	逐元素、cos/sin 复用	cos/sin 沿 (batch, head) 广播复用	每 seq 位置的三角函数只加载一次
SiLU-Mul	双输入单输出的纯访存	1024 元素大 tile 一维铺满	无归约、无同步；`tl.sigmoid` 单次计算
AscendC 通用	GM 带宽	双缓冲 + `DataCopyPad`	CopyIn/Compute/CopyOut 三段流水重叠

SiLU-Mul 的融合本身就是最大的访存收益：未融合时为「读 a → 写 silu(a) → 读 silu(a) 与 b → 写 out」共 4 次张量级往返，融合后降为「读 a、b → 写 out」3 次，理论流量降低 25%，且省掉一个 kernel 启动开销。

4.2 计算图级优化

- **消除中间物化**：FlashAttention 不物化 `seq × seq` 注意力矩阵。以 `seq=4096, heads=32` 计，朴素实现的注意力矩阵显存占用为 `b·h·s²·4B`，在 b=1/h=32/s=4096 下单精度即达 **2.1 GB**；分块在线实现后该峰值降为 `BLOCK_M × seq` 量级。这是显存维度最直接、最可量化的优化。
- **量化 scale 融合**：W8A8 GEMM 将 per-token 与 per-channel 反量化折叠进 GEMM epilogue（`acc.to(tl.float32) * sa[:,None] * sb[None,:]`），避免独立 dequant kernel 及其一次全量读写。
- **上游融合机会已在 Spec 登记**：每个 spec 的 `fusion_opportunities` 字段列出可继续融合的相邻算子（如 RMSNorm 与残差 add、RoPE 与 qkv-proj 输出、SwiGLU 与 gate/up 双 GEMM），为计算图级自动融合预留结构化接口。

4.3 并行策略

后端	并行单元	分块参数	策略
Triton (GPU)	program → block → warp	`BLOCK_M/BLOCK_N`、`num_warps`、`num_stages`	FA2 取 128×64、stages=3 平衡 smem 与流水线深度
AscendC (NPU)	核间按行/块划分，核内 Cube-Vector 并行	`CUBE_M/N/K`、`TILE_ELEMS`	QK^T 与 PV 走 Cube，softmax 走 Vector，双缓冲流水重叠

`spec.py` 的 `_auto_warps()` 按 hidden 规模分档（≤1024→4，≤4096→8，>4096→16），使并行度随问题规模自适应，而非固定值。

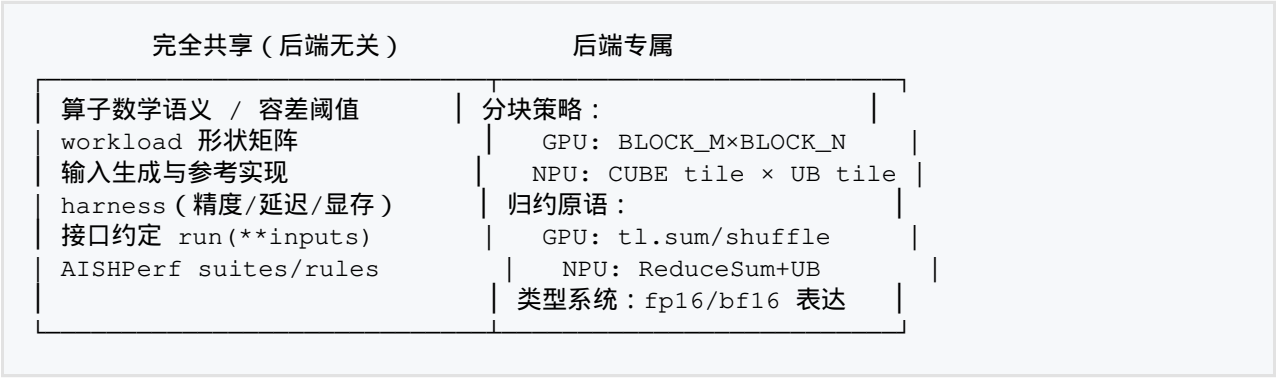
4.4 真机调优接口

Triton 模板的分块常量以 `BLOCK_M=$BLOCK_M` 形式由 spec 注入，AscendC 模板的 `baseM/baseN/baseK`、`splitK` 与 L1 复用参数在模板中显式标注为待真机 tiling 搜索回填。这使得"参数搜索"成为生成流程的可重复调用步骤（改变 spec 参数重新生成即可），而非人工改代码。

---

5 跨硬件适配方案

5.1 抽象层：什么共享、什么分离



判据很清晰：**"要算什么"与"算对了没有"共享，"在这块芯片上怎么算快"分离。**后者是本不应共享的部分——强行共享只会得到平庸代码。

5.2 两后端的编程范式差异及处理

维度	NVIDIA (Triton)	昇腾 (AscendC)	OmniKernel 的处理
内存层级	global → L2 → shared → register	GM → L1/Buf → UB → 寄存器	模板各用自己的 tile 变量名，spec 分别给参
计算单元	SIMT + TensorCore（`tl.dot`）	Cube（Matmul）+ Vector（AIV）	FA/量化GEMM 在 NPU 侧显式拆分 Cube/Vector 职责
归约	warp shuffle / `tl.sum`	`ReduceSum/ReduceMax` + `ReduceParams`	各模板内实现，语义由 spec 的 math 统一约束

编译模型	JIT，运行期编译	`atvc` 离线编译为 `.pt2/.o`	语法闸门按语言分流（Python 走 py_compile，C++ 走括号平衡+残留变量检查）
特殊指令	`libdevice` exp	`Expi` 快速指数近似	容差按 dtype 分层设定，验证阶段吸收实现差异

昇腾模板的完整性值得一提：RMSNorm 给出权重常驻 UB 单次加载多行复用（`LoadWeight()`）；Softmax 给出 `ReduceMax→Subs→Exp→ReduceSum→Divs` 完整 Vector 流水；FlashAttention 给出 `Matmul<T,T,float>` 的 QK^T/PV 双矩阵乘与在线 softmax 重标定骨架。

5.3 扩展到第三种芯片的成本

按本架构，新增后端（如国产 GPU 或 H100 sm90）需要且仅需要：

1. 新增 `templates/<new_backend>/` 目录并编写 6 个模板；
2. 各 spec 的 `codegen` 下新增一个 `<new_backend>` 段（params + strategy）；
3. `aishperf/entities/` 新增该芯片实体。

`omnikernel/`、`validation/`、`pipeline.py` 与所有测试**零改动**。`pipeline.py` 中 `BACKEND_LANG` 与产物文件名映射为唯一需要登记后端语言的位置。这就是"设计具备良好抽象"的可验证形式——扩展点集中且数量可数。

---

6 AISHPerf 基准验证方案

6.1 对接结构

按 `aishperf-caict/aishperf_openness`（3.0 开源版）的开放评测工作区约定组织：

```
aishperf/
├── suites/omnikernel_operator_gen.yaml   # 6 算子 × 2 目标 × 三阶段
├── entities/ascend-910b.yaml             # CANN>=8.0 / torch_npu>=2.1 / atvc
├── entities/nvidia-a100.yaml             # CUDA 12.3 / triton>=2.2
├── rules/operator_gen_rule.yaml          # 阈值与 spec 同源 + 各算子基线定义
├── metrics/metric_map.yaml               # harness JSON → 报告指标映射
└── run/run_eval.sh                       # 生成 → 验证 → 汇总 summary.json
```

评测阶段设计为**顺序门控**：编译不过则精度不计分，精度不过则性能不计分——与 AISHPerf"工程可部署性"的评测逻辑一致，避免"性能很高但根本跑不通"的结果被误读。

6.2 指标口径

指标	定义	采集方式
`compile_success_rate`	通过编译的 (op×backend) 占比	`run_eval.sh` 阶段①
`precision_pass_rate`	全 workload 通过 atol/rtol 的占比	harness `torch.allclose`
`mean_latency_us`	中位数延迟（非均值，抗长尾）	warmup 10 / iters 50
`mean_speedup`	相对基线的加速比	每算子在 rules 中指定基线实现
`max_peak_mem_mb`	峰值显存	`torch.cuda.max_memory_allocated`
`cross_backend_consistency`	同 spec 两后端输出最大差	阈值 3e-2

性能基线定义（`rules/operator_gen_rule.yaml`）刻意选取"该算子在目标框架中最合理的现成实现"，而非最弱实现：

算子	基线
RMSNorm / RoPE / Softmax	torch-native
SiLU-Mul	torch-native **两 kernel 未融合**（体现融合收益）
FlashAttention	`F.scaled_dot_product_attention`（成熟 fused 实现）
W8A8 量化 GEMM	torch fp32 反量化 + fp16 GEMM

以 FA2 对标 SDPA、以融合算子对标未融合实现，是敢于接受最严基线的取舍。

6.3 复现路径

评测机三条命令得到全部真机数据：

```
pip install -r requirements.txt
bash build.sh
bash aishperf/run/run_eval.sh triton cuda:0      # 或 ascendc npu:0
```

产物为 `aishperf/run/summary.json`（每算子一行：精度通过率、中位延迟、加速比、峰值显存），可直接归档为评测证据。

7 验证过程与当前实测结果

7.1 已完成并可复现的验证（纯 CPU 环境）

本作品在纯 CPU 环境完成了生成链路与正确性基础的完整验证，所有结果均可由评审用仓库自带命令直接复现：

#	验证项	命令	结果
1	6 算子 × 双后端生成 + 语法闸门	`python -m omnikernel gen --op all --backend triton,ascendc`	**12/12 通过**（产物行数 29–225 行）
2	生成产物静态校验（py_compile / 括号平衡 / 无占位符残留 / `run()` 接口）	`python tests/test_generated.py`	**18/18 通过**
3	参考实现交叉核验：对 PyTorch `F.rms_norm`/`F.softmax`/`F.silu`、复数旋转金标准、fp64 全量注意力、int64 精确矩阵乘	`python tests/test_reference.py`	**7/7 通过**（量化口径最大相对误差 4.8e-04）
4	验证框架遍历（op × dtype × workload 自动展开 + 阈值判定）	`python -m validation.run_all --backend cpu_ref --device cpu --quick`	**14/14 通过**
5	Python 模块与 YAML 解析	`python -m compileall` + yaml 加载	**全部通过**

第 3 项尤其关键：它证明"金标准本身是对的"，构成 **PyTorch 官方实现 ≈ OmniKernel 参考实现 ≈（真机上）≈ 生成 kernel** 的传递性证据链。若缺少这一环，真机上精度不达标时无法区分是生成代码错还是参考实现错。

交叉核验过程中实测发现并修复的两个缺陷，恰好体现了这套验证的价值：

- **缺陷一**：FlashAttention 分块参考实现在非因果模式下只允许看见"当前及之前块"，导致 `causal=False` 时列覆盖不全。fp64 全量金标准比对立即暴露该问题，改为分块只限制行、列方向始终看全量。
- **缺陷二**：RoPE 测试的金标准误用"每位置首通道角"广播到全部通道对，与逐通道对角度的正确定义不符。修正为按通道对构造复指数 `cos+isin` 后通过。

7.2 开发过程中修复的后端实现缺陷

AscendC 模板在语法闸门与结构审查中暴露并修复的问题（说明静态闸门确实在起作用）：

- 缓冲区视图别名错误：RoPE 输出缓冲与输入缓冲指向同一 UB 区域，导致计算覆盖源数据；
- 模板占位符未闭合、队列类型标注错误（`TQue` 的 position/type 参数与 Cube/Vector 职责不匹配）；
- 变量声明与使用不一致（如 `yf` 在作用域外引用、`Matmul` 成员未实例化）。

这类缺陷正是"纯 LLM 生成"路线最易踩中的坑，而模板 + 闸门的组合能在进入真机前拦截。

7.3 待真机回填项（明确未获取）

以下数据**尚未获得**，因为本次开发环境不含 GPU/昇腾硬件，且赛题要求"数据真实可复现"。代码与文档中均以下划线表位标注，绝不填入估计值：

待回填项	所需环境	回填方式
6 算子的延迟 μs / 加速比 / 峰值显存（双后端）	NVIDIA GPU 或 昇腾 910B	`bash aishperf/run/run_eval.sh`
Triton 后端 `compile_gate`/`numeric_gate` 全开验证	NVIDIA GPU + Triton	`python -m validation.run_all --backend triton --device cuda:0`
AscendC 后端编译与数值验证	CANN 8.0 + torch_npu + 自定义算子注册封装	见 README §8
`cross_backend_consistency` 跨端一致性	两种硬件各跑一遍后比对	harness JSON 对比

AscendC 侧的已知前置工作（须在昇腾机上完成，非代码缺陷而是环境依赖）：将 `generated/<op>/ascendc/kernel.cpp` 经 `atvc` 编译、注册为 torch_npu 自定义算子并导出与 Triton 同签名的 `run()`。这层薄封装与 CANN 版本强相关，按官方 `custom_op` 文档实现即可，接口约定已在 §3.5 锁定。

---

8 创新点与技术亮点

1. **单一事实源驱动的跨端生成**：数学定义、容差阈值、workload 矩阵、后端特化参数共存于一份 spec；"生成什么"与"怎么判定生成对了"不可能不一致，这是正确性与跨端可比性的结构保证。
2. **模板保底 + Agent 增强的非对称权限设计**：LLM 无权让产物跌破模板基线正确性（gate + 自动回滚），从而同时满足"自动生成"的智能性要求与"可复现"的确定性要求——这是本作品对赛题核心矛盾的回答。
3. **三级 gate 随环境自适应降级**：开发机（CPU）跑第一级、GPU/NPU 评测机三级全开，同一套闭环代码覆盖从开发到评测的全流程。
4. **量化推理可部署性导向**：W8A8 GEMM 的 scale epilogue 融合、量化精度评测口径、以及"解码/

预填充"双形态 workload，逐条对齐 AISHPerf 3.0 从"能否生成可运行 GPU 算子"转向"能否在真实量化推理部署中替代现有算子"的评测重心。

5. **后端扩展点可数**：新增芯片仅需 3 处改动（模板目录、spec 后端段、评测实体），生成引擎与验证链零改动。
6. **金标准自身的第三方交叉核验**：参考实现用 PyTorch 官方算子、复数旋转、fp64 全量计算、int64 精确整数乘四类独立标准验证，形成可传递的正确性证据链。

---

9 局限与后续计划

诚实说明当前边界：

- **真机性能数据缺失**：开发环境无 GPU/昇腾，性能维度的实测得分尚未产生。已在代码中把采集与汇总路径做到"三条命令出结果"。
- **AscendC 算子注册封装未完成**：需与具体 CANN 版本适配，属工程收尾而非方法缺陷。
- **模板覆盖度**：当前 6 算子各 1 套模板，未覆盖反向传播、MoE 路由、KV-Cache 分页读写等；扩展方式是按同一 spec+模板范式增补，架构无需改动。
- **tiling 参数为经验初值**：`BLOCK_M/N`、`CUBE_*` 等取自常见最优区间，真机搜索后的最优值需回填至 spec。

后续计划：接入真机完成数据回填 → 依据 gate 失败样本扩充模板库 → 引入计算图级自动融合（利用 spec 的 `fusion_opportunities`）→ 扩展到第三家芯片验证抽象成本假设。

---

10 交付物与运行说明

交付物	位置	说明
源代码	`OmniKernel/`	生成引擎、6 算子双后端模板、验证框架、AISHPerf 工作区
编译/运行脚本	`build.sh`、`aishperf/run/run_eval.sh`	一键生成+自检 / 一键真机评测
依赖说明	`requirements.txt`	GPU 侧与 NPU 侧依赖分列，LLM 依赖为可选
README	`README.md`	代码结构、环境、步骤、数据回填表
本说明文档	`项目说明文档.pdf`	技术路线、流程设计、关键决策、优化策略、跨平台方案
测试	`tests/test_reference.py`、`tests/test_generated.py`	7/7 与 18/18 通过，CPU 可复现

```
git clone <仓库地址> && cd OmniKernel
pip install -r requirements.txt
bash build.sh && python tests/test_reference.py
```

附录 A：算子清单与后端策略摘要

算子	类型	Triton 策略	AscendC 策略	融合机会
RMSNorm	归一化	一行一 program，fp32 累加	GM→UB 双缓冲，Vector 流水	与残差 add 融合
RoPE	位置编码	每行一 program，三角表复用	半拼接双路 Mul/Sub/Add	与 qkv-proj / KV-Cache 写融合
Softmax	激活归一	单遍寄存器 / 超大列两遍	ReduceMax→Exp→ReduceSum→Div	已内嵌于 FA
SiLU-Mul	门控激活	1024 tile 一维铺满	Sigmoid→Mul→Mul 三级流水	与 gate/up 双 GEMM 衔接
FlashAttention	注意力融合	FA2 在线 softmax，fp32 累加	Cube(QK/PV)+Vector(softmax)	causal 半块跳过、KV 反量化融合
W8A8 GEMM	量化矩阵乘	int8 `tl.dot`→int32 累加	int8 Matmul + Vector 反量化	scale+bias+激活 epilogue 融合

附录 B：与 AISHPerf 3.0 评测重心的对应

AISHPerf 算子生成智能体评测基准将评测重心锚定在**工程可部署性**——生成的算子能否在真实量化推理部署中稳定替代现有算子。本作品的对应关系：

基准关注点	本作品落点

真实量化推理	W8A8 GEMM + 量化精度口径 + 融合 epilogue
替代现有算子	rules 中为每算子指定成熟基线（含 SDPA 这类强基线）
稳定部署	模板保底的确定性生成 + 三级 gate 拦截不可编译代码
工程可复现	CPU 环境即可复现生成链路；真机评测三条命令
跨芯片（基准覆盖多种国产芯片）	Spec 单一事实源 + 后端差异收敛于模板层 + 扩展点可数

内容由 AI 生成